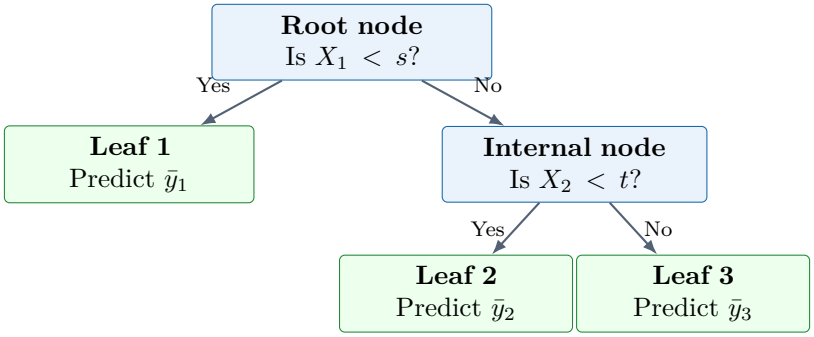
Introduction to Decision Trees

– Regression and Classification



Detailed lecture notes with mathematical derivations,
visual intuition, worked numericals, and exam questions

Central idea

A decision tree recursively partitions the feature space into regions.
Each region returns a simple prediction: a numerical value for regression
or a class label (and possibly class proportions) for classification.

Contents

1 The Basics of Decision Trees **1**

1.1 What is a decision tree? . . . 1

1.2 Tree vocabulary . . . 1

1.3 How a prediction is produced . . . 2

1.4 Why trees are piecewise models . . . 2

2 Regression Trees **3**

2.1 Purpose and intuition . . . 3

2.2 Motivating example: baseball-player salaries . . . 3

2.3 The optimal prediction within a leaf . . . 4

2.4 Residual sum of squares (RSS) . . . 4

2.5 Recursive binary splitting . . . 4

2.5.1 Mathematical formulation of a candidate split . . . 5

2.5.2 Worked numerical: compare candidate splits . . . 5

2.5.3 Why is the procedure called greedy? . . . 6

2.6 Stopping rules and overfitting . . . 6

2.7 Cost-complexity pruning . . . 6

2.7.1 Choosing tree size using cross-validation . . . 7

3 Classification Trees **8**

3.1 What changes in classification? . . . 8

3.2 Class proportions and predicted class . . . 8

3.3 Classification error rate . . . 8

3.4 Gini impurity . . . 8

3.5 Entropy . . . 9

3.6 Worked impurity example . . . 9

3.7 How a classification split is scored . . . 9

3.8 Why not use classification error to grow every tree? . . . 10

3.9 Categorical predictors . . . 10

4 Regression Trees vs Classification Trees **11**

4.1 A unifying view 11

5 Trees Versus Linear Models **12**

5.1 Mathematical forms 12

5.2 Visual comparison of decision boundaries 12

5.3 Detailed comparison 12

5.4 When should each be preferred? 13

6 Advantages, Limitations, and Practical Guidance **14**

6.1 Advantages 14

6.2 Limitations 14

6.3 Practical modelling checklist 14

7 Exam Practice: Numericals **16**

7.1 Numerical 1: Optimal regression prediction 16

7.2 Numerical 2: Parent RSS and split improvement 16

7.3 Numerical 3: Classification impurity 16

7.4 Numerical 4: Weighted Gini reduction 17

7.5 Numerical 5: Cost-complexity threshold 17

7.6 Numerical 6: Impurity after a proposed split 17

7.7 Numerical 7: Classification error versus Gini 17

8 True/False Questions with Explanations **18**

9 Edge and Conceptual Questions **19**

10 Final Formula Sheet and Revision Map **21**

10.1 Must-know equations 21

10.2 Last-minute mental checklist 21

Chapter 1

The Basics of Decision Trees

1.1 What is a decision tree?

A **decision tree** is a supervised learning model that maps an input vector

$$\boldsymbol{x} = (x_1, x_2, \ldots, x_p)$$

to a prediction by applying a sequence of simple tests. At every internal node, a feature is compared with a threshold or a category-based rule. The test determines which branch the observation follows next.

Trees are used for both:

- **Regression:** the response Y is quantitative, e.g. salary, price, or temperature.
- **Classification:** the response Y is categorical, e.g. spam/not spam or disease/no disease.

Key idea

A tree does not fit one global equation to all observations. It divides the predictor space into regions and gives each region its own prediction.

1.2 Tree vocabulary

Root node The first node, initially containing all training observations.

Internal node A node at which a feature test splits observations.

Split The rule used to divide one node into two child nodes.

Cutpoint A threshold s, for example $X_j < s$.

Branch An edge representing one outcome of a split.

Leaf / terminal node A final node that returns the prediction.

Depth Number of edges between the root and a node.

Region A subset of predictor space corresponding to a leaf.

1.3 How a prediction is produced

1. Start at the root node.
2. Evaluate its condition using the new observation's feature values.
3. Follow the branch corresponding to the condition's outcome.
4. Continue until a terminal node is reached.
5. Return the value or class stored at that leaf.

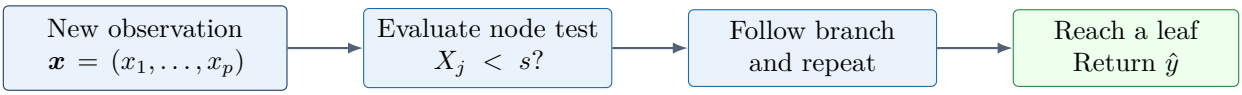


Figure 1.1: The prediction path through a decision tree.

1.4 Why trees are piecewise models

Suppose a feature X is split at s. The model can assign one prediction to observations with $X < s$ and a different prediction to observations with $X \geq s$. A second split can subdivide one of these groups further. Therefore the fitted function is **piecewise constant** for a standard regression tree.

$$\hat{f}(\boldsymbol{x}) = \sum_{m=1}^{M} c_m \, \mathbf{1}(\boldsymbol{x} \in R_m),$$

where $R_1, \ldots, R_M$ are disjoint regions that cover the predictor space and c_m is the prediction for region R_m. Since each observation belongs to exactly one leaf region, exactly one indicator in the sum equals one.

Chapter 2

Regression Trees

2.1 Purpose and intuition

A regression tree predicts a numerical response. Its leaves contain numerical values, commonly the mean response of the training observations that reach the leaf. The training process attempts to create regions whose response values are internally similar.

2.2 Motivating example: baseball-player salaries

Consider predicting a baseball player's log-salary from:

- `Years`: years played in the major leagues;
- `Hits`: hits made in the previous year.

A fitted tree may first split at `Years` < 4.5. The left leaf contains players with four or fewer years. Players with at least five years move right, where a second split may test `Hits` < 117.5.

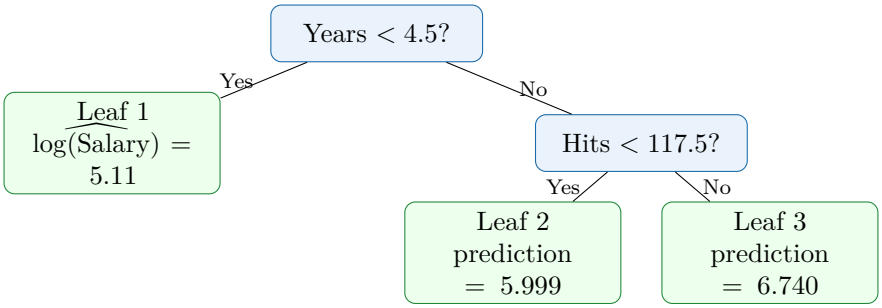


Figure 2.1: Illustrative regression tree. Each leaf predicts the mean log-salary of the training observations in that region.

If the response was log-transformed, a prediction on the log scale can be exponentiated to obtain a value on the original scale. However, exponentiating the mean log-response is not generally identical to predicting the arithmetic mean salary; this distinction matters when interpreting results.

2.3 The optimal prediction within a leaf

Suppose a leaf R_m contains n_m observations with responses y_i. We assign a constant prediction c to every observation in this leaf. Under squared-error loss, choose c to minimize

$$Q(c) = \sum_{i:\boldsymbol{x}_i \in R_m} (y_i - c)^2.$$

Deriving the mean prediction

Differentiate with respect to c:

$$\frac{dQ}{dc} = \sum_{i:\boldsymbol{x}_i \in R_m} 2(y_i - c)(-1) = -2 \sum_{i:\boldsymbol{x}_i \in R_m} (y_i - c).$$

Set the derivative equal to zero:

$$-2 \sum_{i:\boldsymbol{x}_i \in R_m} (y_i - c) = 0 \quad \Longrightarrow \quad \sum_{i:\boldsymbol{x}_i \in R_m} y_i - n_m c = 0.$$

Therefore

$$\boxed{\hat{c}_m = \bar{y}_m = \frac{1}{n_m} \sum_{i:\boldsymbol{x}_i \in R_m} y_i.}$$

The second derivative is

$$\frac{d^2 Q}{dc^2} = 2n_m > 0,$$

so the stationary point is the unique minimum whenever $n_m > 0$.

Exam checkpoint

For squared-error loss, the mean is optimal. If the loss were absolute error instead, the median would minimize the sum of absolute deviations. Do not confuse these two results.

2.4 Residual sum of squares (RSS)

For a tree with M terminal regions, the training residual sum of squares is

$$\boxed{\mathrm{RSS}(T) = \sum_{m=1}^{M} \sum_{i:\boldsymbol{x}_i \in R_m} (y_i - \bar{y}_m)^2.}$$

Each residual is $e_i = y_i - \hat{y}_i$. Squaring penalizes large deviations, and summing measures the total within-leaf variation around the leaf means.

Key idea

A split is attractive for regression when it separates observations into child nodes whose responses are more homogeneous, thereby reducing the combined RSS.

2.5 Recursive binary splitting

Searching over every possible partition of feature space is computationally infeasible. A common approach is **recursive binary splitting**: start with one region, split it into two, and then repeatedly split one of the resulting regions.

2.5.1 Mathematical formulation of a candidate split

For feature X_j and cutpoint s, define

$$R_L(j,s) = \{\boldsymbol{x} : x_j < s\}, \qquad R_R(j,s) = \{\boldsymbol{x} : x_j \geq s\}.$$

For each candidate split, calculate the mean response in each child. Its total RSS is

$$\mathrm{RSS}(j,s) = \sum_{i:\boldsymbol{x}_i \in R_L(j,s)} (y_i - \bar{y}_L)^2 + \sum_{i:\boldsymbol{x}_i \in R_R(j,s)} (y_i - \bar{y}_R)^2.$$

The selected split is

$$\boxed{(j^*, s^*) = \arg\min_{j,s} \mathrm{RSS}(j,s).}$$

Equivalently, because the parent RSS is fixed while comparing splits, choose the split that gives the largest reduction in RSS.

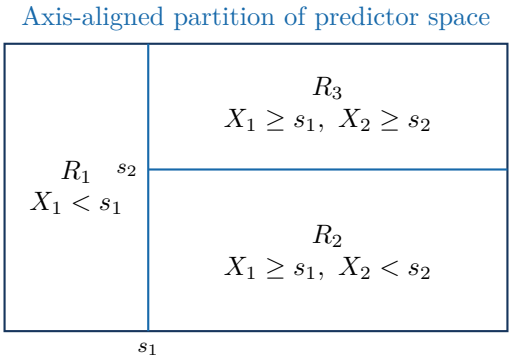


Figure 2.2: A sequence of binary splits creates non-overlapping rectangular regions in two-dimensional feature space.

2.5.2 Worked numerical: compare candidate splits

Consider responses $y = [1, 2, 8, 9]$. Compare two candidate partitions.

Split A versus Split B

Split A: left $[1, 2]$, right $[8, 9]$.

$$\bar{y}_L = 1.5, \quad RSS_L = (1-1.5)^2 + (2-1.5)^2 = 0.5,$$

$$\bar{y}_R = 8.5, \quad RSS_R = (8-8.5)^2 + (9-8.5)^2 = 0.5.$$

Hence $RSS_A = 1$.
Split B: left $[1, 8]$, right $[2, 9]$. Both means are 4.5 and 5.5:

$$RSS_L = (1-4.5)^2 + (8-4.5)^2 = 24.5,$$

$$RSS_R = (2-5.5)^2 + (9-5.5)^2 = 24.5.$$

Hence $RSS_B = 49$. Split A is preferred because $1 < 49$.

2.5.3 Why is the procedure called greedy?

It is **top-down** because it starts at the root, **binary** because each split creates two children, and **recursive** because the process repeats within child regions. It is **greedy** because each step chooses the best split at that moment, without searching every possible future tree. Consequently, it does not guarantee a globally optimal tree.

2.6 Stopping rules and overfitting

If splitting continues until leaves are extremely small, the tree may memorize noise in the training data. Training RSS usually decreases as the tree grows, but test error can rise. Common constraints include:

- maximum depth;
- minimum number of observations required to split a node;
- minimum number of observations in a leaf;
- minimum required improvement in the split criterion.

Stopping early is simple, but it can be shortsighted: a split with modest immediate improvement may enable a valuable split later. A widely used alternative is to grow a large tree and then prune it.

2.7 Cost-complexity pruning

Let T be a subtree of a large tree T_0. Cost-complexity pruning minimizes

$$\boxed{C_\alpha(T) = \mathrm{RSS}(T) + \alpha|T|,}$$

where $|T|$ is the number of terminal nodes and $\alpha \geq 0$ controls the complexity penalty.

$\alpha = 0$ The leaf-count penalty vanishes; among subtrees, training RSS favors the more flexible tree.

Moderate α
A split must reduce RSS enough to justify the added leaves.

Large α Simpler trees are favored because every terminal node incurs a larger penalty.

Comparing two candidate subtrees

Suppose tree A has $RSS_A = 40$ and $|A| = 5$, while tree B has $RSS_B = 46$ and $|B| = 2$. Their penalized costs are

$$C_\alpha(A) = 40 + 5\alpha, \qquad C_\alpha(B) = 46 + 2\alpha.$$

Tree B is preferred when

$$46 + 2\alpha < 40 + 5\alpha \iff 6 < 3\alpha \iff \boxed{\alpha > 2.}$$

At $\alpha = 2$, the two costs are equal.

2.7.1 Choosing tree size using cross-validation

1. Grow a large tree on the training data.
2. Generate a nested sequence of pruned subtrees for different α values.
3. Split the training set into K folds.
4. For each fold and each α, fit on $K-1$ folds and compute validation MSE on the held-out fold.
5. Average the validation errors across folds and choose the α with the lowest average error (or use a justified simpler-tree rule).
6. Fit the chosen model using the full training data, then evaluate once on the untouched test set.

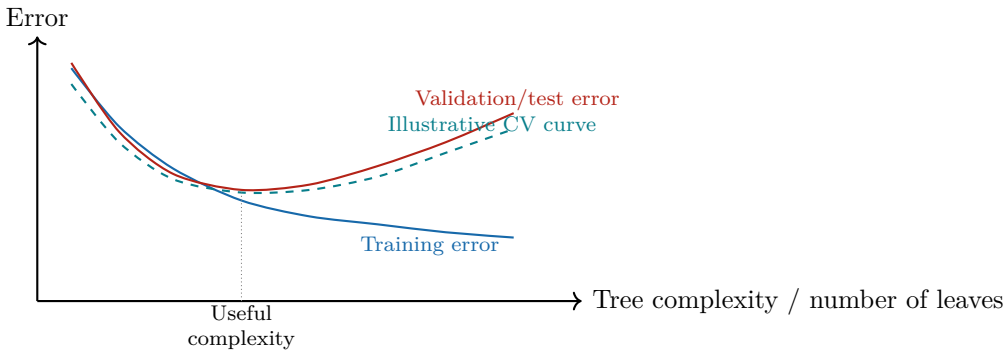


Figure 2.3: Conceptual bias–variance pattern: training error tends to decrease with complexity, while validation/test error may rise after overfitting begins. The curves are schematic, not measured data.

Exam checkpoint

Do not select tree size by training RSS alone. The training criterion rewards extra flexibility; cross-validation estimates how well that flexibility generalizes.

Chapter 3

Classification Trees

3.1 What changes in classification?

A classification tree predicts a qualitative response. Its branching structure is similar to a regression tree, but the target is a class label rather than a numerical response. A leaf usually predicts the most common training class in that region.

3.2 Class proportions and predicted class

Suppose a leaf R_m contains n_m training observations and K possible classes. The estimated proportion belonging to class k is

$$\boxed{\hat{p}_{mk} = \frac{1}{n_m} \sum_{i : \boldsymbol{x}_i \in R_m} \mathbf{1}(y_i = k).}$$

The predicted class is

$$\boxed{\hat{y}_m = \operatorname*{arg\,max}_{k \in \{1,\dots,K\}} \hat{p}_{mk}.}$$

The proportions $\hat{p}_{m1}, \dots, \hat{p}_{mK}$ describe the class mixture within the leaf. They can be used as rough probability estimates, but estimates from small leaves may be noisy or poorly calibrated.

3.3 Classification error rate

If the majority class proportion in a node is $\max_k \hat{p}_{mk}$, the fraction of observations misclassified by predicting that class for everyone is

$$\boxed{E_m = 1 - \max_k \hat{p}_{mk}.}$$

For example, if a node contains 7 positive and 3 negative observations, its majority class is positive and its classification error is $1 - 0.7 = 0.3$.

3.4 Gini impurity

The Gini impurity is

$$\boxed{G_m = \sum_{k=1}^{K} \hat{p}_{mk}(1 - \hat{p}_{mk}) = 1 - \sum_{k=1}^{K} \hat{p}_{mk}^2.}$$

For a binary class distribution $(p, 1-p)$,

$$G = 1 - (p^2 + (1-p)^2) = 2p(1-p).$$

Gini is zero for a pure node and largest when the class proportions are equal. For K classes, its maximum is $1 - \frac{1}{K}$.

Why does Gini measure impurity?

If one class has proportion 1, then

$$G = 1 - 1^2 = 0.$$

For two equally common classes, $p_1 = p_2 = 1/2$:

$$G = 1 - \left(\frac{1}{4} + \frac{1}{4}\right) = \frac{1}{2}.$$

The more mixed the node, the larger the Gini impurity. The quantity $1 - \sum_k p_k^2$ can also be interpreted as the probability that two independent labels drawn from the node's class distribution differ.

3.5 Entropy

Entropy is another measure of node impurity:

$$\boxed{D_m = -\sum_{k=1}^{K} \hat{p}_{mk} \log \hat{p}_{mk},}$$

with the convention $0 \log 0 = 0$. If base-2 logarithms are used, entropy is measured in bits. It equals zero for a pure node and is maximized by a uniform class distribution.

For the binary distribution $(0.7, 0.3)$,

$$D = -0.7 \log_2(0.7) - 0.3 \log_2(0.3) \approx 0.881 \text{ bits}.$$

3.6 Worked impurity example

A node with 7 positives and 3 negatives

The class proportions are $p_+ = 0.7$ and $p_- = 0.3$.

$$\begin{aligned} \text{Classification error} &= 1 - \max(0.7, 0.3) = 0.30, \\ G &= 1 - (0.7^2 + 0.3^2) = 1 - (0.49 + 0.09) = 0.42, \\ D &= -0.7 \log_2(0.7) - 0.3 \log_2(0.3) \approx 0.881. \end{aligned}$$

A pure node containing only positive examples has classification error 0, Gini 0, and entropy 0.

3.7 How a classification split is scored

For a parent node containing n observations, suppose a candidate split creates left and right children with n_L and n_R observations. The weighted child impurity is

$$I_{\text{split}} = \frac{n_L}{n} I(L) + \frac{n_R}{n} I(R),$$

where I can be Gini or entropy. The impurity reduction is

$$\boxed{\Delta I = I(\text{parent}) - I_{\text{split}}.}$$

A larger positive reduction indicates a better split under that criterion.

Weighted Gini calculation

A parent has 20 observations, 10 from each class. Thus

$$G_P = 1 - (0.5^2 + 0.5^2) = 0.5.$$

A split creates:

- Left child: 7 class A and 1 class B, so $n_L = 8$.
- Right child: 3 class A and 9 class B, so $n_R = 12$.

For the left child, proportions are $(0.875, 0.125)$:

$$G_L = 1 - (0.875^2 + 0.125^2) = 0.21875.$$

For the right child, proportions are $(0.25, 0.75)$:

$$G_R = 1 - (0.25^2 + 0.75^2) = 0.375.$$

The weighted impurity is

$$G_{\text{split}} = \frac{8}{20}(0.21875) + \frac{12}{20}(0.375) = 0.3125.$$

Therefore the Gini reduction is

$$\boxed{\Delta G = 0.5 - 0.3125 = 0.1875.}$$

3.8 Why not use classification error to grow every tree?

Classification error only changes when the majority class or its proportion changes enough to alter the error fraction. Gini and entropy respond more smoothly to shifts in class proportions. Thus Gini and entropy are typically more sensitive criteria for growing a tree. Classification error remains an intuitive measure of misclassification and may be used to evaluate a final tree.

Common trap

A split can improve Gini or entropy even when both children predict the same majority class as the parent. The split may make each child more confident/pure without changing the predicted label.

3.9 Categorical predictors

A categorical predictor can be split by assigning some categories to one branch and the remaining categories to the other. For example, a test may separate one value of a stress-test variable from the remaining values. A split is useful when it produces child nodes with more homogeneous outcomes. High-cardinality categorical features need care because many possible groupings can encourage overfitting.

Chapter 4

Regression Trees vs Classification Trees

Aspect	Regression tree	Classification tree
Target	Quantitative response Y	Categorical response Y
Leaf prediction	Mean response under squared-error loss	Majority class; may also report class proportions
Growth criterion	Reduction in RSS / MSE	Reduction in weighted Gini or entropy
Leaf quality	Low within-leaf response variation	Low class mixture / high purity
Typical metrics	MSE, RMSE, MAE, R^2	Accuracy, precision, recall, F1, ROC-AUC, log loss
Example	Predict a house price	Predict whether a transaction is fraudulent

Table 4.1: The main distinction is the response type and the criterion used to assess candidate splits.

4.1 A unifying view

Both types of trees:

- recursively divide the feature space using binary tests;
- assign each observation to a terminal region;
- use a local summary of training observations in that region;
- can overfit if allowed to grow too large;
- benefit from validation, cross-validation, and pruning/complexity control.

The target and loss determine what the local summary and split criterion should be.

Chapter 5

Trees Versus Linear Models

5.1 Mathematical forms

A linear regression model assumes a global linear form:

$$\boxed{f(\boldsymbol{x}) = \beta_0 + \sum_{j=1}^{p} \beta_j x_j.}$$

A regression tree instead uses a piecewise-constant function:

$$\boxed{f(\boldsymbol{x}) = \sum_{m=1}^{M} c_m \mathbf{1}(\boldsymbol{x} \in R_m).}$$

A linear model has one set of coefficients for the whole feature space. A tree assigns separate predictions to separate regions.

5.2 Visual comparison of decision boundaries

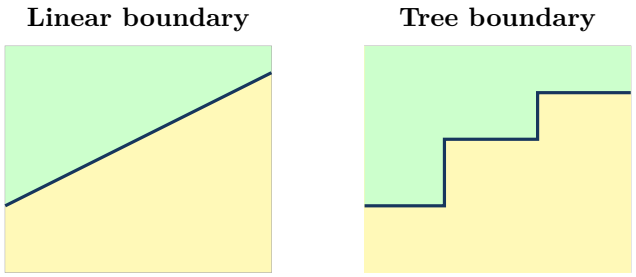


Figure 5.1: A linear classifier can represent a diagonal boundary directly. A standard tree approximates a diagonal boundary with multiple axis-aligned regions.

5.3 Detailed comparison

Property	Linear models	Decision trees
Shape	Global linear relationship unless features are transformed	Piecewise-constant regression function; nonlinear patterns can be represented by regions
Interactions	Must be specified or engineered in a standard linear model	Can arise naturally through sequential splits
Boundary	Linear in the modeled feature space for standard linear/logistic models	Axis-aligned rectangular regions for standard binary splits
Interpretation	Coefficients summarize conditional associations under the model	A sequence of if–then rules describes local regions
Stability	Often relatively stable, particularly with regularization	A small change in data can change early splits and the whole tree
Extrapolation	Can continue a trend beyond the observed feature range, sometimes dangerously	Standard regression leaves predict constants; extrapolation is usually poor
Scaling	Often important for regularized models and distance-based preprocessing	Usually not necessary for threshold-based splitting
Best fit	Approximately linear effects or a need for a compact global equation	Rule-like interactions, thresholds, and complex nonlinear structure

5.4 When should each be preferred?

Choose based on the data and objective

- Prefer a linear model when the relationship is approximately linear, a compact coefficient-based summary is important, or a stable global effect estimate is desired.
- Consider a tree when threshold rules and interactions matter, nonlinear structure is expected, or explanations in the form of decisions are useful.
- Neither dominates universally. Compare validation/test performance and consider interpretability, stability, and the cost of errors.

Common trap

Trees are not automatically better for nonlinear data. Smooth or diagonal relationships can require many rectangular leaves. Conversely, a plain linear model can fail when the true relationship is strongly rule-based or highly nonlinear.

Chapter 6

Advantages, Limitations, and Practical Guidance

6.1 Advantages

- **Interpretability:** small trees can be explained as a sequence of questions.
- **Nonlinearity:** sequential splits represent threshold effects without manually choosing polynomial terms.
- **Interactions:** the effect of a later feature can depend on earlier decisions.
- **Mixed feature types:** numerical and categorical predictors can be handled by suitable implementations.
- **Little scaling requirement:** standard threshold-based trees generally do not require standardized features.

6.2 Limitations

- **Instability:** small data changes can produce a substantially different tree.
- **Overfitting:** deep trees and tiny leaves can capture noise.
- **Greedy search:** recursive splitting does not guarantee the globally optimal tree.
- **Piecewise-constant regression:** predictions jump at split boundaries and do not naturally extrapolate trends.
- **Accuracy:** a single tree often performs less accurately than well-tuned ensembles such as random forests or gradient-boosted trees.

6.3 Practical modelling checklist

1. Define the prediction target and choose a metric that matches the real objective.
2. Keep the test set untouched during model selection.
3. Tune maximum depth, minimum leaf size, and pruning strength using validation or cross-validation.
4. For imbalanced classification, consider stratified splits and inspect precision, recall, F1, and the confusion matrix rather than relying only on accuracy.
5. Check leaf sizes and class proportions; very small leaves can yield unstable estimates.

6. Avoid leakage: feature selection, imputation, and other learned preprocessing must be fitted only on training folds.
7. Compare the final tree with a sensible baseline model.

Chapter 7

Exam Practice: Numericals

7.1 Numerical 1: Optimal regression prediction

A terminal node contains responses $3, 5, 6, 10$. Find the prediction that minimizes squared error and compute the RSS.

Solution

The mean is

$$\bar{y} = \frac{3+5+6+10}{4} = 6.$$

Therefore

$$RSS = (3-6)^2 + (5-6)^2 + (6-6)^2 + (10-6)^2 = 9+1+0+16 = \boxed{26}.$$

7.2 Numerical 2: Parent RSS and split improvement

A parent node has responses $2, 4, 6, 8$. Its mean is 5, so

$$RSS_{\text{parent}} = 9+1+1+9 = 20.$$

A proposed split creates left $[2, 4]$ and right $[6, 8]$. The child means are 3 and 7, and the combined child RSS is $2+2=4$. Thus

$$\boxed{\text{RSS reduction} = 20-4 = 16.}$$

7.3 Numerical 3: Classification impurity

A node has 8 examples of class A and 2 of class B. Compute classification error, Gini impurity, and entropy in bits.

Solution

$$p_A = 0.8, \qquad p_B = 0.2.$$

$$E = 1 - 0.8 = \boxed{0.2}.$$

$$G = 1 - (0.8^2 + 0.2^2) = 1 - (0.64 + 0.04) = \boxed{0.32}.$$

$$D = -0.8\log_2(0.8) - 0.2\log_2(0.2) \approx \boxed{0.722 \text{ bits}}.$$

7.4 Numerical 4: Weighted Gini reduction

A parent node has equal proportions of two classes, so its Gini is 0.5. A split creates a left node of size 8, with proportions $(0.875, 0.125)$, and a right node of size 12, with proportions $(0.25, 0.75)$. The child impurities are $G_L = 0.21875$ and $G_R = 0.375$.

$$G_{\text{split}} = \frac{8}{20}(0.21875) + \frac{12}{20}(0.375) = 0.3125,$$

so the reduction is

$$\boxed{0.5 - 0.3125 = 0.1875.}$$

7.5 Numerical 5: Cost-complexity threshold

Tree A has RSS 40 and 5 leaves; Tree B has RSS 46 and 2 leaves. Determine when B is preferred.

$$C_\alpha(A) = 40 + 5\alpha, \qquad C_\alpha(B) = 46 + 2\alpha.$$

B is preferred if $46 + 2\alpha < 40 + 5\alpha$, which gives

$$\boxed{\alpha > 2.}$$

7.6 Numerical 6: Impurity after a proposed split

A parent node contains 6 positive and 4 negative examples. A split creates: left: 4 positive and 1 negative; right: 2 positive and 3 negative. Compute the parent Gini and weighted child Gini.

Solution

Parent proportions are $(0.6, 0.4)$:

$$G_P = 1 - (0.6^2 + 0.4^2) = 0.48.$$

Left proportions are $(0.8, 0.2)$, so $G_L = 1 - (0.64 + 0.04) = 0.40$. Right proportions are $(0.4, 0.6)$, so $G_R = 1 - (0.16 + 0.36) = 0.48$. The weighted child impurity is

$$G_{\text{split}} = \frac{5}{10}(0.40) + \frac{5}{10}(0.48) = 0.44.$$

Hence the Gini reduction is $\boxed{0.48 - 0.44 = 0.04}$.

7.7 Numerical 7: Classification error versus Gini

Compare a node with proportions $(0.6, 0.4)$ and one with proportions $(0.9, 0.1)$.

$$E_{0.6/0.4} = 0.4, \quad G_{0.6/0.4} = 1 - (0.36 + 0.16) = 0.48,$$

$$E_{0.9/0.1} = 0.1, \quad G_{0.9/0.1} = 1 - (0.81 + 0.01) = 0.18.$$

The second node is purer by both measures. In particular, Gini responds to changes in class proportions even when the majority class remains the same.

Chapter 8

True/False Questions with Explanations

1. A regression tree predicts the mean response of training observations in the reached leaf under squared-error loss.
True. The mean minimizes the within-leaf sum of squared deviations.

2. A deeper regression tree must have lower test MSE.
False. Training RSS usually falls as splits are added, but test error can rise because of overfitting.

3. Gini impurity and entropy are zero for a pure classification node.
True. A pure node has one class proportion equal to one and all others zero.

4. Classification error is generally more sensitive than Gini or entropy when growing a tree.
False. Gini and entropy respond more smoothly to changes in class proportions.

5. Every decision tree creates a diagonal decision boundary.
False. Standard trees use axis-aligned splits; multiple splits can approximate diagonal boundaries by a staircase.

6. Cost-complexity pruning penalizes the number of terminal nodes.
True. The objective includes $\alpha|T|$.

7. At $\alpha = 0$, the complexity penalty has no effect.
True. The objective reduces to training RSS for regression.

8. Greedy recursive binary splitting guarantees the globally optimal tree.
False. It optimizes each local split rather than searching all possible trees.

9. A leaf with class proportions 0.6 and 0.4 predicts the class with proportion 0.6.
True. The majority class is selected, although the leaf is not pure.

10. Features must always be standardized before fitting a decision tree.
False. Threshold-based tree splits generally do not require standardization.

11. A split can improve Gini while leaving the predicted class unchanged.
True. It may create children with more extreme class proportions but the same majority label.

12. A regression tree naturally extrapolates a linear trend beyond the training range.
False. Standard regression trees predict a constant value within each terminal region.

Chapter 9

Edge and Conceptual Questions

Q1. Can two different trees have identical training error but different test error?
Yes. They may partition sparsely sampled regions differently or fit different patterns of noise. Training performance alone does not determine generalization.

Q2. Can a split improve Gini without changing the majority class?
Yes. A split may make each child node purer while leaving the majority label unchanged. For example, a positive-majority parent can split into two positive-majority children with different degrees of purity.

Q3. Why is cross-validation preferable to training RSS for choosing tree size?
Training RSS tends to favor large trees. Cross-validation estimates performance on held-out observations and helps select complexity that generalizes.

Q4. What happens when α becomes extremely large in cost-complexity pruning?
The penalty on terminal nodes dominates the fit term, so a very small tree, often the root-only tree, is selected.

Q5. Why can a tree represent an L-shaped boundary naturally?
A sequence of axis-aligned splits can create rectangular regions that combine to form an L-shaped decision region.

Q6. Why can a diagonal boundary require many tree leaves?
A standard tree cannot represent the diagonal with a single split. It approximates the boundary using a staircase of horizontal and vertical segments.

Q7. Does lower training Gini guarantee lower test classification error?
No. Gini is a local training criterion. Test performance also depends on sample size, noise, pruning, and distribution shift.

Q8. What is the difference between a predicted class and a class probability?
The predicted class is the class with the largest estimated proportion. Class proportions describe the training-label mix in a leaf; they are not automatically perfectly calibrated probabilities.

Q9. Can both children of a split predict the same class? Why retain such a split?
Yes. The split may improve Gini or entropy and produce more reliable class proportions, even if the majority label stays the same. Whether it should be retained depends on validation performance and pruning.

Q10. Why might a linear model be preferred if a tree has slightly better validation accuracy?
A linear model may provide a more stable global summary, interpretable coefficients, or better extrapolation. The decision should account for uncertainty in the validation estimate and the costs of model complexity.

Q11. Does a smaller RSS always imply a better regression model?
Not by itself. RSS is measured on the data used to fit the model. A more complex tree can lower training RSS while increasing test error.

Q12. Why does a tree have high variance?
Early splits determine the subsequent regions. A small change in the data can change an early split, which changes all downstream partitions.

Chapter 10

Final Formula Sheet and Revision Map

10.1 Must-know equations

Regression leaf prediction: $\hat{y}_m = \bar{y}_m = \dfrac{1}{n_m} \displaystyle\sum_{i:\boldsymbol{x}_i \in R_m} y_i$

Regression RSS: $\mathrm{RSS}(T) = \displaystyle\sum_{m=1}^{M} \sum_{i:\boldsymbol{x}_i \in R_m} (y_i - \bar{y}_m)^2$

Candidate split: $R_L = \{\boldsymbol{x} : x_j < s\}, \qquad R_R = \{\boldsymbol{x} : x_j \geq s\}$

Classification error: $E_m = 1 - \max_k \hat{p}_{mk}$

Gini impurity: $G_m = 1 - \displaystyle\sum_{k=1}^{K} \hat{p}_{mk}^2$

Entropy: $D_m = -\displaystyle\sum_{k=1}^{K} \hat{p}_{mk} \log \hat{p}_{mk}$

Weighted child impurity: $I_{\text{split}} = \dfrac{n_L}{n} I(L) + \dfrac{n_R}{n} I(R)$

Cost-complexity pruning: $C_\alpha(T) = \mathrm{RSS}(T) + \alpha |T|$

Linear regression: $f(\boldsymbol{x}) = \beta_0 + \displaystyle\sum_{j=1}^{p} \beta_j x_j$

Regression tree: $f(\boldsymbol{x}) = \displaystyle\sum_{m=1}^{M} c_m \mathbf{1}(\boldsymbol{x} \in R_m)$

10.2 Last-minute mental checklist

1. **Regression or classification?** Identify the response type first.
2. **What is predicted at a leaf?** Mean for squared-error regression; majority class for classification.
3. **How is a split selected?** Minimize child RSS for regression; reduce weighted Gini/entropy for classification.

4. **Why prune?** To control complexity and improve generalization.
5. **How choose tree size?** Use validation or cross-validation, not training error alone.
6. **Tree versus linear model?** Trees capture rule-like nonlinear structure; linear models represent global linear trends compactly.

Closing perspective

A decision tree is a collection of local rules. Its strength is that complex patterns can be represented through a sequence of understandable splits. Its weakness is that the learned structure can be unstable and overly complex. The practical objective is not to create the deepest possible tree; it is to select a tree whose structure is useful and whose predictive performance generalizes to unseen observations.